

seen to turn a three-point DT into a four-point DT, then into a five-point DT, and so on until we've added all of our points.

In [Listing 7-3](#), we can combine the code we wrote earlier to create a function called `gen_delaunay()`, which takes a set of points as its input and outputs a full DT.

```
def gen_delaunay(points):
    delaunay = [points_to_triangle([-5, -5], [-5, 10], [10, -5])]
    number_of_points = 0

    while number_of_points < len(points): ❶
        point_to_add = points[number_of_points]

        delaunay_index = 0

        invalid_triangles = [] ❷
        while delaunay_index < len(delaunay):
            circumcenter, radius =
triangle_to_circumcenter(delaunay[delaunay_index])
            new_distance = get_distance(circumcenter, point_to_add)
            if(new_distance < radius):
                invalid_triangles.append(delaunay[delaunay_index])
                delaunay_index += 1

        points_in_invalid = [] ❸
        for i in range(0, len(invalid_triangles)):
            delaunay.remove(invalid_triangles[i])
            for j in range(0, len(invalid_triangles[i])):
                points_in_invalid.append(invalid_triangles[i][j])
        points_in_invalid = [list(x) for x in set(tuple(x) for x in
points_in_invalid)]

        for i in range(0, len(points_in_invalid)): ❹
            for j in range(i + 1, len(points_in_invalid)):
                #count the number of times both of these are in the bad triangles
                count_occurrences = 0
                for k in range(0, len(invalid_triangles)):
                    count_occurrences += 1 * (points_in_invalid[i] in
invalid_triangles[k]) * \
                    (points_in_invalid[j] in
invalid_triangles[k])
                if(count_occurrences == 1):
                    delaunay.append(points_to_triangle(points_in_invalid[i],
```